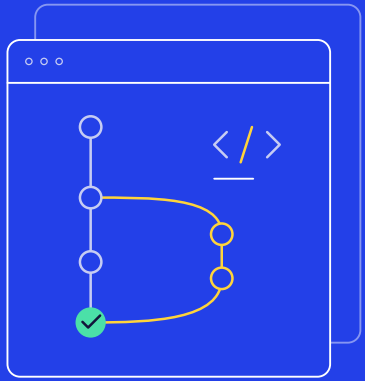
WEEK 1 / LAB 01

MLOps: Machine Learning Operations

Shared Repository, Git Workflow & Code Quality Checks

Start your team's Bike Demand workflow.



PART 1

Set up the repo and team workflow



Agree how the team will work.

01 → 02 → 03

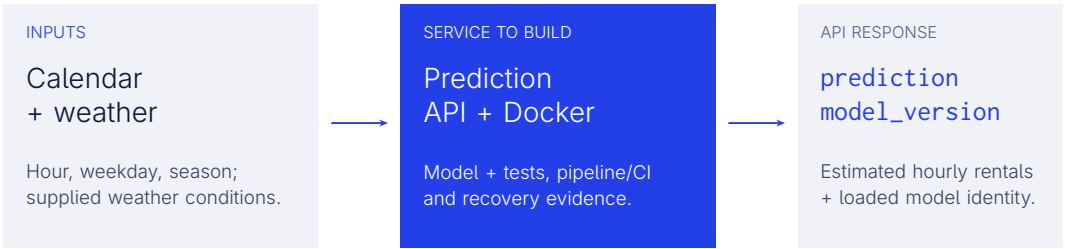
Get access

Read the starter

Agree the workflow

One Bike Sharing project, built across the course

Estimate total hourly rentals from the supplied calendar and weather inputs.



Design overview — local CPU core; optional cloud, no bonus marks.

Lab 01 starts the shared workflow. Lab 02 introduces data and training.

Eight labs build the same service

Weeks 1–4 develop the system; Week 5 verifies and presents it.

Week 1	Lab 1 / Repository, peer review, checks Lab 2 / Data, split, baseline, tracking	m1
Week 2	Lab 3 / Dependency lock, Docker, clean rerun Lab 4 / Prediction API, validation, tests	m2
Week 3	Lab 5 / Deployment, logs, model identity Lab 6 / GitHub Actions, regression tests	m3
Week 4	Lab 7 / Training pipeline, quality gate Lab 8 / Incident diagnosis, verified rollback	m4

Week 5 / Clinic, final verification and presentations; no new lab or report.

m1–m4 are ungraded checkpoints: link check results and remaining blockers.

Final handover: one service, with evidence

Week 5: GitHub Release `v1.0.0` + team presentation (60%).

Release `v1.0.0`

Run
Code/tests, trained model,
locked setup + Docker instructions.

Verify
Link revision, configuration and model
to metrics, test/CI results and recovery.

Hand over
README, recovery runbook
and traceable release notes.

Presentation flow

Explain
User + scope; data split, model
and evaluation against the baseline.

Demonstrate
Valid API call + loaded model version;
invalid-input behaviour.

Support with evidence
Tests/CI, gate and recovery evidence;
each member explains their contribution.

No need to rerun full training, builds or CI during the presentation.

Two assessment components; no separate lab grade

The team project and each student's own reflection contribute different evidence.

60%

Team project + presentation

One integrated system, verification evidence and the final team presentation.

4 × 10%

Individual reports

One private report after each of Weeks 1–4; 40% in total.



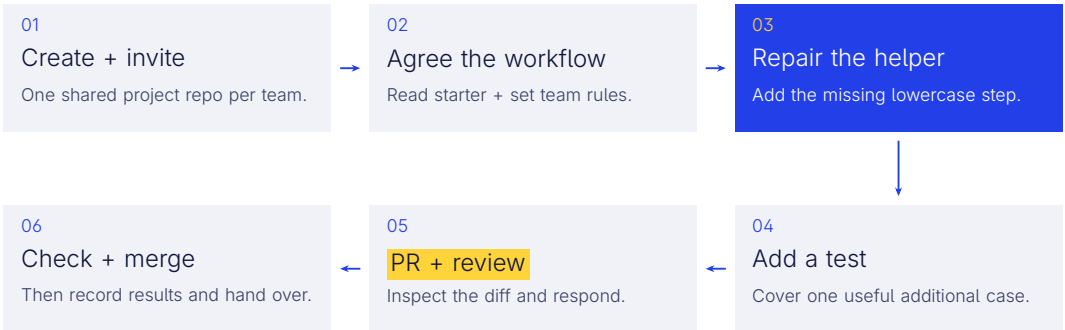
Team component

Four equally weighted reports

Team checkpoints m1–m4 support feedback; they carry no marks.

Today: one small change, from access to handover

The team owns the change, review and merge. We do not train a model today.



Form a team and verify repository access

Teams of 3–4 choose how to work; no self-approval.

01 **Choose how to work**

Split the real work; pair when useful.

02 **Review each change**

Another member reviews and approves the PR.

03 **Make contributions visible**

Each member: an authored commit/PR or useful review.

Each member checks access and runs preflight.
If pairing, switch driver and navigator during the task.

STUDENT-OWNED REPOSITORIES

One project repo per team.
One private report repo per student.

Invite the instructor; register links.

QR placeholder

Register your team and repository links.
Use the announced course route.

Read the starter before changing it

Separate what the approved package supplies from what your team must complete.

GITHUB / SCREENSHOT PLACEHOLDER

Student repository + README

Repository files and README.
Source, tests and configuration.
Team worksheet and working notes.

Placeholder – not execution evidence.

Find these parts

`src/`: the helper
`tests/`: the behaviour contract
Configuration + team worksheet

Your contribution

The lowercase fix, one useful test,
a reviewed PR and actual evidence.

Keep code, tests, PRs and handover
in the same team repository.

Before coding: agree how your team will work

Write three short agreements in the shared repository; use them in your first PR.

01

Project scope

In `README.md`:
user + goal;
inputs + output;
scope + non-goals;
how success is checked.

02

Contribution rules

In `CONTRIBUTING.md`:
branch + commit naming;
local checks;
PR + peer review;
who merges and when.

03

Team plan

In brief working notes:
availability + help route;
next steps + blockers;
handover: who did what.

Done when a teammate can pick up a task and know how to get it reviewed and merged.

Choose your conventions; keep the course scope, peer review and checks.

PART 2

From a shared repo to the first change

Complete one small task that a teammate can test, inspect and review.

Small scope. Checkable outcome.



Lab 01 task: complete the team slug

Add the missing lowercase conversion in `normalize_team_slug`.

```
" Team Blue "  ──────────→  "team-blue"
```

Requirements

Keep trimming; collapse whitespace to hyphens; reject blank names.
Do not rewrite the whole helper.

Read tests; add one useful case.
Do not change assertions, skip tests or disable checks to hide the fault.

Deliverables

Code + test on a branch.
One pull request.
A different teammate's review and your response.

Completion criteria

Rerun checks on this revision.
Team review, then team merge.
Record the checked commit, results and blockers.

Remote CI blocked?
Say so explicitly.

Check setup before interpreting a test failure

Use the approved starter from its project root, not this slide workspace.

APPROVED STARTER / PREFLIGHT

```
uv sync --locked
uv run --locked python -m bike_demand.preflight
```

Expected exercise failure

Only the named lowercase assertion fails.
That result alone does not validate the environment.

Setup or distribution problem

Sync/import failures or additional failing tests: investigate.

TERMINAL / SCREENSHOT PLACEHOLDER

Setup + preflight result

Environment and command outcome.
Setup checks and exercise tests.
Any error that needs investigation.

Placeholder – not execution evidence.

Missing lock or setup blocked? Ask for help; do not replace the environment.

Make a small, reviewable Git change

Branch, edit, inspect and check before you commit and push.

START A TASK BRANCH

```
git status --short
git switch -c lab-01-workflow
```

01 Edit the helper + add a test

02 Inspect the diff

03 Run the local checks

AFTER EDITING + CHECKING

```
git diff
git add src/bike_demand/team.py tests/test_team.py
git commit -m "Normalize team labels"
git push -u origin lab-01-workflow
```

Stage only intended files; use the approved access route.
Never expose credentials in commands, commits or screenshots.

Branch → Code + test → Diff → Checks → Commit → Push

Run local checks, read the result, then recheck

Testing, linting and formatting answer different questions. A command is not a result.

01 RUFF
Code quality

Is the code consistent?

02 INFRA
Setup health

Is the supplied setup healthy?

03 LAB 1
Behaviour

Does the label follow its contract?

TERMINAL / READ EACH CHECK'S OWN SIGNAL

```
uv run --locked ruff check .
uv run --locked ruff format --check .
uv run --locked pytest -q -m infra
uv run --locked pytest -q -m lab1
```

Then run all tests

AFTER THE FIX

```
uv run --locked pytest -q
```

No secrets, environments or generated artifacts in Git.

Record actual outcomes; expectations are not measurements.

A pull request is a review conversation

Another member approves; checks pass on the current revision before merge.

01 Describe what changed and how you checked it.

02 Read code + test together. Give feedback, then approve.

03 Respond and rerun checks. Then the team merges.

GITHUB / PLACEHOLDER

PR description

Explain the helper fix, added test and actual checks. Use the same repository + PR.

Placeholder – not execution evidence.

GITHUB / PLACEHOLDER

Code + test diff

Helper fix and added test. Inspect changed behaviour. Check that the diff stays small.

Placeholder – not execution evidence.

GITHUB / PLACEHOLDER

Review + team merge

Feedback, response and approval. Checks on the current revision. The resulting merged commit.

Placeholder – not execution evidence.

Remote checks belong to a specific revision

Local success does not show that GitHub ran the same checks.

GITHUB / SCREENSHOT PLACEHOLDER

PR checks / Actions run

Tested revision and check names.
Actual conclusions and job logs.
Evidence for the current PR.

Placeholder – not execution evidence.

Before merging

01 Match the check to the revision you intend to merge.

02 Read the actual outcome; open logs when a check fails.

03 Confirm review + response and record any unresolved blocker.

Class procedure is not automatic GitHub merge enforcement.

Remote run blocked? Record why; do not claim a pass.

Read the lowercase failure, repair it, then recheck

The existing assertion shows which part of the behaviour contract is missing.

REAL LOCAL CAPTURE / 1 OCTOBER 2026

```
F....... [100%]
>       assert normalize_team_slug("  Team Blue  ") == "team-blue"
E       AssertionError: assert 'Team-Blue' == 'team-blue'
E         - team-blue
E         + Team-Blue
1 failed, 7 passed in 0.02s
```

OBSERVED LOCALLY

1 failed

7 passed

Only the lowercase assertion failed.

System interpreter only. Not setup validation. Not remote CI.

Read the assertion

Actual: `Team-Blue`
Expected: `team-blue`

Repair the conversion

Keep whitespace and empty-name behaviour.

Rerun + record

Checks, tested commit and any blocker.

PART 3

From a repaired change to completion evidence

Show what actually happened.



01 Check results

02 Record blockers

03 Handover

One team record; each student explains their work

Link to shared evidence. Explain your own contribution; do not copy the team worksheet.

TEAM / LAB 1 WORKSHEET

`reports/lab-01.md`

EVIDENCE

Code/test PR + checked revision;
each member's commit or useful review.

RECORD

Passed checks + peer approval;
handover: who did what + blockers.

ASSESSMENT

Shared, ungraded worksheet.
m1 follows review of both labs.

INDIVIDUAL / PRIVATE WEEK 1 REPORT

`reports/week-1.md`

EVIDENCE

Your contribution + evidence;
one decision or debugging example.

RECORD

Private repo you create;
invite the instructor; submit on main.

ASSESSMENT

10% of the course grade.
No report PR or CI; keep it private.

Continue in the same repository

Before handover:
check Lab 01 completion.

LAB 01 DONE WHEN

Each member confirms a working environment.
A short merge agreement is in the repo.
One PR with code + test has peer approval, passing checks and a merged commit.
Handover: who did what + commit/review links.

m1 follows both Week 1 labs; it is ungraded.
No weekly Release or separate Lab 01 grade.

Lab 02: data and a baseline

Add the supplied data/training extension.
Validate, split, train and track.
Change who implements and reviews.

Your Week 1 report

`reports/week-1.md`

Private repo you create; submit on main.
No report PR / CI. Covers Labs 01–02.
10%; Sunday 23:59.
Use the published date and time zone.